Pedestrian Footfall Monitoring in Public Spaces Using Computer Vision

DevScope

2025/2026

1220804 Susana Loureiro

ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pedestrian Footfall Monitoring in Public Spaces Using Computer Vision

DevScope

2025/2026

1220804 Susana Loureiro

Bachelor's Degree in Computer Engineering

June/2026

ISEP Supervisor: **Carlos Ramos**

External Supervisor: **David Mota**

Statement of Integrity

I hereby declare having conducted this academic work with integrity.

I have not plagiarised or applied any form of undue use of information or falsification of results along the process leading to its elaboration.

Therefore the work presented in this document is original and authored by me, having not previously been used for any other end.

I further declare that I have fully acknowledged the Code of Ethical Conduct of P.PORTO.

ISEP, June 6, 2026

Abstract

Monitoring pedestrian footfall in urban public spaces is a growing need for entities managing urban infrastructure. Traditional manual counting approaches prove insufficient given the variability and continuity of pedestrian movement, motivating the development of an automated real-time monitoring system carried out as part of a curricular internship at DevScope.

The solution is built on a three-layer architecture: people detection and counting through computer vision using artificial intelligence techniques and the OpenCV library, data persistence in a non-relational database, and dashboard visualisation. The You Only Look Once (YOLO) model in its YOLOv8s version with tuned inference parameters was adopted as the final approach, following a comparison between seven approaches, including YOLOv8n and YOLOv8s variants with and without fine-tuning and the Azure AI Vision service. Quantitative evaluation was conducted over 450 manually annotated frames using MAE, MAPE, and exact match rate. The tuned model achieved a MAE of 2.31 and an exact match rate of 24.40% on public videos, and a MAE of 2.10 in the real-world evaluation. The solution meets all defined objectives and incorporates facial anonymisation mechanisms, providing a foundation for future work namely in the collection of training data directly from the production environment and in the integration of multiple cameras, towards footfall monitoring and real-time analysis of more complex urban spaces.

Keywords: Computer Vision, Artificial Intelligence, YOLO Model, OpenCV, Footfall Monitoring, Real-time Analysis

Resumo

A monitorização da afluência pedonal em espaço público urbano constitui uma necessidade crescente para entidades que gerem infraestruturas urbanas. As abordagens tradicionais de contagem manual revelam-se insuficientes face à variabilidade e continuidade do fluxo de pessoas, o que motivou o desenvolvimento de um sistema automatizado de monitorização em tempo real, realizado no âmbito de um estágio curricular na DevScope.

A solução assenta numa arquitetura de três camadas: deteção e contagem de pessoas através de visão por computador com recurso a técnicas de inteligência artificial e à biblioteca OpenCV, persistência dos dados em base de dados não relacional, e visualização em *dashboard*. Foi adotado o modelo YOLO na sua versão YOLOv8s com parâmetros de inferência ajustados, após comparação entre sete abordagens, incluindo variantes YOLOv8n e YOLOv8s com e sem *fine-tuning* e o serviço Azure AI Vision. A avaliação quantitativa foi realizada sobre 450 *frames* anotados manualmente, com base nas métricas MAE, MAPE e taxa de correspondência exata. O modelo ajustado obteve um MAE de 2.31 e uma taxa de correspondência exata de 24.40% nos vídeos públicos, e um MAE de 2.10 em ambiente real. A solução cumpre os objetivos definidos e incorpora mecanismos de anonimização facial, constituindo uma base para trabalho futuro nomeadamente na recolha de dados de treino diretamente no ambiente de produção e na integração de múltiplas câmaras, com vista à monitorização de afluência e análise em tempo real de espaços urbanos de maior complexidade.

Palavras-chave: Visão por Computador, Inteligência Artificial, Modelo YOLO, OpenCV, Monitorização de Afluência, Análise em Tempo Real

Acknowledgements

I would like to thank DevScope for the opportunity to be part of a real-world project and for the supportive environment provided throughout the internship. In particular, I would like to thank David Mota for his close guidance and for his willingness to help, which went far beyond what was expected. From technical guidance to practical support with the configuration of the capture environment, he was always there when I needed him most.

I would also like to thank Professor Carlos Ramos, my ISEP supervisor, for his guidance throughout the preparation of this report and for the perspective he brought on artificial intelligence and computer vision. His insights not only enriched this project but also shaped the way I approach the field.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Urban public space is a shared resource whose efficient use depends on knowledge of the flow of people passing through it. The growing population density in urban centres and the need to monitor high-footfall areas render traditional manual counting approaches manifestly insufficient. This reality demands smarter solutions capable of providing continuous, real-time information about pedestrian flow behaviour.

In this context, real-time pedestrian footfall monitoring emerges as a growing need for entities managing urban infrastructure. Knowing how many people circulate in a given area, at what times of day, and with what footfall patterns enables informed decisions about safety, accessibility, and urban space planning. People-counting technology, combined with data processing and visualisation, represents a concrete response to this challenge.

It is within this context that the present report was developed, as part of a curricular internship at DevScope, a Portuguese software consultancy. The proposed project aimed to develop a pedestrian footfall monitoring system for public space, capable of counting people in real time and generating relevant statistics for the analysis of urban flow behaviour.

1.1 Background/Context

DevScope is a Portuguese software consultancy with clients worldwide. As part of its internal innovation initiatives, the company proposed the development of a pedestrian footfall monitoring system applied to an urban public space near its premises. The variability of people flow in this space throughout the day makes real-time monitoring especially relevant for footfall pattern analysis and the extraction of useful statistics.

1.2 Problem Description

The absence of real-time pedestrian footfall information makes it difficult to understand the patterns of public space usage. Without footfall data, it is not possible to identify peak periods of people concentration, nor to anticipate situations of overcrowding or underutilisation of the space. Furthermore, the lack of a historical record prevents the analysis of trends and informed decision-making about urban space management.

1.2.1 Objectives

The main objective of this project was to develop a pedestrian footfall monitoring system for public space, capable of identifying and counting people in real time through computer vision. Beyond real-time detection, the system aims to produce a historical footfall record that

allows the extraction of relevant statistics for space analysis, namely the number of people present throughout the day and the identification of peak footfall periods. These data are intended to support decisions about the management and planning of urban public space.

1.2.2 Approach

The developed solution is built on a three-layer architecture: real-time people detection and counting through computer vision, persistent storage of footfall data, and statistical dashboard visualisation. For detection, a deep learning-based approach was adopted to ensure accurate counting of individuals over time. The collected data are persisted in a non-relational database and made available through an interactive dashboard, enabling historical analysis of pedestrian flow.

1.2.3 Contributions

The main contribution of this project is the development of an integrated real-time pedestrian footfall monitoring system applied to an urban public space context. From an analytical standpoint, the system enables the identification of footfall patterns and peak periods of people concentration. From a privacy standpoint, the solution incorporates identity anonymisation mechanisms, ensuring compliance with data protection principles in public space. Finally, the modular architecture adopted allows the solution to be replicated or adapted to other spaces and contexts with similar needs.

1.2.4 Work Planning

The work was planned in five main phases, distributed across the internship period, as presented in the following table.

Phase	**Period**
DevScope RampUp	23/2 – 6/3
State of the art presentation – Start of report writing	7/3 – 31/3
Solution prototype development	1/4 – 2/5
Prototype testing and validation	3/5 – 23/5
Final presentation of the developed solution and completed report	24/5 – 5/6

TABLE 1.1: Work planning

1.3 Report Structure

In addition to the introduction, this dissertation contains 4 more chapters. Chapter 2 describes the state of the art and presents related work. Chapter 3 presents the analysis carried out for the implementation of the solution described in Chapter 4. The fifth and final chapter describes the conclusions drawn.

Chapter 2

State of the Art

This chapter presents a review of the literature and existing technologies in the domain of people counting through computer vision. The main approaches to people detection are analysed, together with the most relevant tracking algorithms and related works that formed the basis for decisions made in the present project. The aim is to contextualise the current state of knowledge in the field and to justify, based on existing evidence, the technologies selected for the developed solution.

2.1 Computer Vision

Computer vision is a branch of artificial intelligence that enables machines to interpret and understand visual information from the real world, such as images and videos. Its central objective is to replicate the human capacity to extract meaning from visual data, enabling tasks such as object detection, pattern recognition, and motion analysis [1], [2].

In the context of pedestrian footfall monitoring, computer vision emerges as a natural solution to the people-counting problem. Unlike approaches based on other physical sensors, computer vision systems operate from cameras, such as those already present in city infrastructure, making their adoption more economical and scalable [3]. The ability to process video streams in real time enables continuous information about people footfall to be obtained without the need for human intervention.

Recent advances in deep learning, in particular the development of convolutional neural networks (Convolutional Neural Network (CNN)), have significantly boosted the performance of computer vision systems applied to people detection [4], [5].

CNNs are a type of deep neural network specifically designed to process data with spatial structure, such as images. Their architecture is composed of three main types of layers: convolutional layers, pooling layers, and fully connected layers. Convolutional layers apply filters, also called kernels, that slide over the input image and produce feature maps, automatically learning to detect relevant visual patterns. In the early layers, these filters tend to capture low-level features such as edges and textures; in deeper layers, they combine these features to represent more complex patterns such as shapes and object parts. Pooling layers reduce the spatial dimensionality of the feature maps, making the representation more compact and robust to small positional variations. Finally, fully connected layers aggregate the extracted features and produce the final network output, whether a classification or a detection.

The main advantage of CNNs over classical approaches such as Histogram of Oriented Gradients (HOG) lies precisely in their ability to learn visual representations directly from training data, without the need for manual feature definition [6], [7]. These models are

capable of learning complex visual representations from large volumes of data, far surpassing classical approaches based on hand-crafted features such as HOG [7].

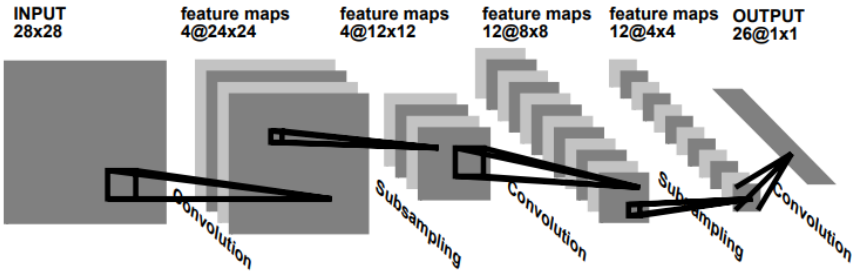

FIGURE 2.1: Typical CNN architecture with convolutional, pooling and fully connected layers. Source: [6]

In the specific context of people detection and counting, computer vision algorithms can be organised into four main categories [8]:

- **Gaussian process regression:** offers low computational complexity, but requires manual parameter tuning and exhibits poor robustness across different scenarios;
- **Frame differencing:** effective in low-density scenes, but highly dependent on a stable camera and a static background;
- **Optical flow:** estimates motion between consecutive frames based on the apparent movement of individual pixels;
- **Object detection and tracking:** offers greater adaptability and robustness compared to the previous categories [8]; this is the approach adopted in the present project.

2.2 People Detection Models

People detection in images and videos can be approached through different model families, each with distinct characteristics in terms of accuracy, speed, and computational complexity.

One of the most classical approaches is HOG combined with Support Vector Machine (SVM). HOG extracts features based on the distribution of intensity gradients in the image, being robust to scale, rotation, and illumination variations [2]. The SVM acts as a classifier, distinguishing regions with and without people based on these features [7].

Despite its simplicity, this approach presents significant limitations in real-time scenarios, being surpassed in speed and accuracy by deep learning-based models [2].

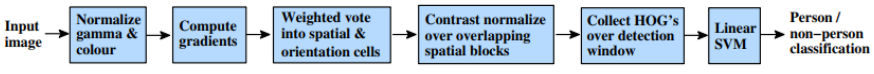

FIGURE 2.2: HOG descriptor pipeline for people detection with SVM classification. Source: [9]

Single Shot MultiBox Detector (SSD) is a one-stage detector that simultaneously predicts the classes of detected objects and the coordinates of their bounding boxes, without the need for a separate region proposal stage [10]. Its SSD MobileNet variant replaces the backbone with a lighter architecture, making it suitable for low-cost hardware such as the Raspberry Pi

[10], [11]. However, SSD presents lower accuracy than You Only Look Once (YOLO) in more complex scenarios [2].

The YOLO family, which we will analyse next, currently represents the state of the art in real-time object detection. Like SSD, YOLO is a one-stage detector, processing the entire image in a single pass of the neural network. Internally, YOLO divides the input image into a grid of cells, predicting for each cell a set of bounding boxes and the probability of each object class. Each bounding box is described by five values: centre coordinates, width, height, and detection confidence score [12].

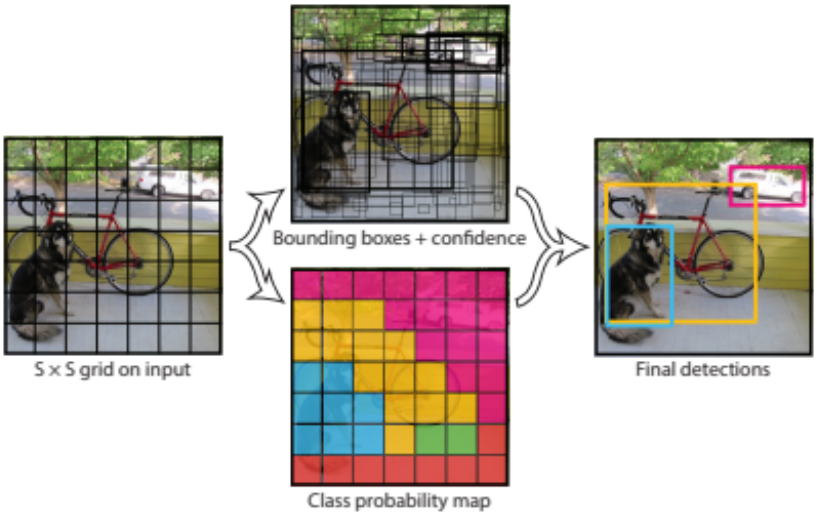

FIGURE 2.3: You Only Look Once version 8 (YOLOv8) architecture, illustrating the division of the image into a grid of cells and the prediction of bounding boxes per cell. Source: [12]

This architecture gives YOLO a significant speed advantage over two-stage approaches such as Region-based Convolutional Neural Network (R-CNN). YOLOv8 introduces architectural improvements that make it more reliable and accurate compared to previous versions, demonstrating superior results to competing models in people-counting tasks [12], and clearly outperforming HOG-SVM in real-time scenarios [2]. Although later versions such as YOLO11 have since been published, YOLOv8 remains a widely validated choice in the literature for this type of application. It is worth noting that the choice of YOLO version has a direct impact on per-frame processing time, so lighter versions such as YOLOv4-tiny may be preferable on more constrained hardware [3].

The performance of detection models is typically evaluated using Mean Average Precision (mAP), calculated as the mean of the Average Precision (AP) values obtained for each class, as expressed in the following equation:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^{N} AP_i \tag{2.1}$$

where N represents the number of classes and AP_i the average precision for class i.

An emerging alternative approach is Vision Transformers (ViT), introduced by Dosovitskiy et al. [13]. Unlike CNNs, which process the image through local convolutions, ViTs divide the

image into fixed-size patches, linearly embed them, add positional embeddings, and process the resulting sequence through a standard Transformer encoder [13]. This self-attention mechanism allows global dependencies between image regions to be captured. When pre-trained on large volumes of data, ViTs achieve competitive results against CNNs on classification tasks while requiring less computational training resources [13]. However, their adoption in real-time people detection systems remains limited in the literature, with YOLO being the predominant choice in the works analysed. From a practical standpoint, Transformer encoder-based ViT models typically exhibit CPU inference latencies in the order of hundreds of milliseconds per image [13], incompatible with the real-time processing requirements defined for this type of system [13].

Considering the approaches analysed, YOLOv8 stands out as the most suitable choice for the present project. HOG-SVM, despite its simplicity, presents significant limitations in real-time scenarios, being surpassed in speed and accuracy by deep learning-based models [2]. SSD MobileNet, although suitable for low-cost hardware, presents lower accuracy in more complex scenarios [2]. YOLOv8, combining real-time processing speed with superior accuracy over the analysed alternatives [12], simultaneously satisfies the performance and accuracy requirements defined for the system. Vision Transformers, despite their potential, remain limited in the literature for real-time detection applications, and do not constitute a viable alternative for the use case under study.

Regarding the use of pre-trained models, the literature recognises that this approach represents a significant advantage in terms of development time and cost [14]. However, its suitability depends on the use case: if the pre-trained model is not sufficiently specific for the context in question, results may be unsatisfactory [14].

2.3 Tracking Algorithms

In computer vision-based people counting systems, detection alone is insufficient to ensure accurate counting. When a camera processes a video stream, the same person appears in multiple consecutive frames, so without a tracking mechanism, each detection would be counted independently, resulting in incorrect counts. Tracking algorithms solve this problem by assigning a unique identifier to each detected person and maintaining that identifier consistently over time [5].

The Simple Online and Real-Time Tracking (SORT) algorithm was one of the first effective approaches for real-time tracking. SORT combines Kalman filtering, to predict the position of the object in the next frame, with the Hungarian algorithm to associate detections with existing trajectories based on the Intersection over Union (IoU) value between bounding boxes [5]. Despite its simplicity and speed, SORT presents limitations in occlusion scenarios, where a person is temporarily obstructed by another object, frequently losing the assigned identifier.

IoU is calculated as the ratio between the intersection area and the union area of two bounding boxes, as expressed in the following equation:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} \tag{2.2}$$

where A and B represent two bounding boxes. An IoU value close to 1 indicates near-total overlap, while a value close to 0 indicates that the boxes do not overlap.

The Kalman filter is a recursive algorithm that estimates the state of a dynamic system from a sequence of noisy measurements [15]. In the context of object tracking, the state corresponds to the position and velocity of each object, and the measurements correspond to the bounding boxes detected in each frame. The algorithm operates in two phases: in the prediction phase, it estimates the expected position of the object in the next frame based on its previous motion; in the update phase, it corrects this estimate based on the new detection obtained. This approach allows intermittent and noisy detections to be handled, making tracking more stable and continuous [15].

DeepSORT emerges as an extension of SORT that addresses precisely this limitation. In addition to the SORT mechanisms, DeepSORT incorporates a visual appearance descriptor generated by a pre-trained convolutional neural network, which extracts visual features from each detected person [5]. This way, the algorithm is capable of re-identifying a person after a period of occlusion, maintaining their original identifier. This capability makes DeepSORT significantly more robust in real-world environments where partial occlusions are frequent [8].

The combination of a detection model with DeepSORT has been widely adopted in the literature for real-time people counting systems, demonstrating consistent results across different contexts, from shopping centres [5] to crowd monitoring spaces [8].

2.4 Datasets

Training and evaluating people detection and tracking models requires annotated datasets with images or videos containing people in different contexts.

The Common Objects in Context (COCO) dataset is one of the most widely used in object detection tasks, containing over 200,000 annotated images with 80 object classes, including the *person* class [16]. YOLOv8 is pre-trained on COCO by default, meaning it already has the ability to detect people without additional training.

The Multiple Object Tracking (MOT) Challenge is the most widely used multiple object tracking dataset in the literature, having been published in successive versions: MOT15, MOT16, MOT17, and MOT20 [17]. MOT17, in particular, is widely used for training and evaluating tracking algorithms such as DeepSORT [15]. The videos contain sequences of pedestrians in public spaces, with bounding box annotations and unique identities per person.

The Large-Scale Overhead Fisheye (LOAF) dataset was specifically designed for people detection and localisation with fisheye overhead cameras, containing over 457,000 people annotations across more than 43,000 frames [18]. Although the overhead perspective is relevant for the present project, the characteristic distortion of fisheye cameras requires additional optical distortion correction processing that does not apply to the use case under study.

AttMOT is a synthetic dataset for pedestrian tracking with additional semantic annotations, such as gender, clothing colour, and body shape, generated in a simulation environment [19]. CrowdTrack, in turn, is a large-scale dataset for tracking in real complex scenarios, filmed mostly from a first-person perspective [17]. Both datasets were designed for outdoor scenarios with high pedestrian density.

TABLE 2.1: Comparison of the main people detection and tracking datasets.

Dataset	Size	Context	Perspective	Relevance
COCO	200k images	Outdoor	Normal	Medium
MOT17	11 sequences	Outdoor	Normal	Medium
LOAF	43k frames	Indoor	*Overhead*	Medium
AttMOT	Synthetic	Outdoor	Normal	Low
CrowdTrack	Large scale	Outdoor	First person	Low

2.5 Related Work

Real-time people counting through computer vision has been widely studied in the literature, with various approaches proposed for different contexts and requirements.

The work by Mokayed [5] is the most similar to the system developed in this project. The authors propose a real-time people detection and counting system applied to shopping centres, combining You Only Look Once version 3 (YOLOv3) with the DeepSORT algorithm for tracking. The system includes a graphical interface with management features and uses an intrusion line to determine entries and exits. Experimental results demonstrate an accuracy of 91.07%, with the system capable of operating in real time using a Graphics Processing Unit (GPU). The use of DeepSORT for tracking and the concern with accurate counting in continuous flow scenarios, where people constantly enter and exit the camera's field of view, are aspects directly relevant to the present work.

The system proposed by Cuong Le [10] is relevant for demonstrating the feasibility of real-time people counting systems using low-cost hardware. The authors use an overhead camera (positioned from a top-down perspective, with the field of view directed vertically towards the ground) combined with the MobileNetv2-SSD model for detection and the SORT algorithm with Minimum Output Sum of Squared Error (MOSSE) for tracking, implemented on a Raspberry Pi. This approach is directly comparable to the capture architecture adopted in the present project, which similarly uses a Raspberry Pi as the capture device, demonstrating that real-time performance is achievable with this type of hardware, albeit with limitations in terms of accuracy under variable illumination conditions.

Shyaa and Hashim [12] and Elaoua et al. [20] specifically explore YOLOv8 for people counting, validating its effectiveness in different scenarios. Shyaa and Hashim demonstrate that YOLOv8 achieves accuracy close to 100% in terms of mAP in surveillance contexts, while Elaoua et al. combine YOLOv8 with region-based analysis for counting entries and exits in defined areas of interest.

Vasantha et al. [2] present an empirical comparison between different approaches for people counting, namely Multi-task Cascaded Convolutional Networks (MTCNN) and YuNet for face detection, and HOG-SVM and YOLOv8 for object detection. The results confirm the superiority of YOLOv8 over HOG-SVM in terms of speed and accuracy in real-time scenarios.

Razzok et al. [15] propose improvements to the DeepSORT algorithm through the introduction of new data association metrics, evaluated with the YOLOv5 model on the MOT17 dataset. The authors demonstrate that replacing the standard cost matrix based on IoU with alternative metrics based on bounding box distances and intersections improves tracking performance across most MOT metrics. This work is relevant for deepening the understanding of DeepSORT's internal workings and for validating its effectiveness in real-time pedestrian tracking scenarios.

The remaining works analysed explore alternative approaches. Curiel et al. [3] use YOLOv4-tiny with fine-tuning for detection at gates, highlighting the importance of the YOLO model version in per-frame processing time. Kumar Singh et al. [11] and Krishnachaithanya et al. [14] adopt the SSD model with MobileNet and centroid tracking, a lighter but less robust approach in occlusion scenarios. Finally, Myint and Sein [7] and Valencia et al. [8] use HOG-SVM and YOLOv4-tiny with DeepSORT, respectively, in smaller-scale contexts and social distancing monitoring.

Chapter 3

Solution Analysis and Design

This chapter presents the problem analysis and the design of the developed solution. Starting from a description of the problem domain, the functional and non-functional requirements of the system are identified, followed by the use cases and the solution architecture. The technical description follows a top-down approach, progressively detailing each component of the solution.

3.1 Problem Domain

The problem domain centres on the real-time monitoring of pedestrian flow in urban public spaces. The variability of the number of people circulating in a given area throughout the day makes it difficult to understand footfall patterns without access to concrete data. In the absence of an automated monitoring system, the analysis of urban flow behaviour depends on manual methods that are time-consuming and difficult to scale.

To understand the domain, Figure 3.1 presents the system's domain model, identifying the main concepts and the relationships between them. The simplicity of the model reflects the nature of the problem: the system monitors a single type of entity (people) in a single spatial context, with no complex relationships between entities and no multiple business actors. This simplicity is intentional and appropriate for the use case, and does not constitute a limitation of the model but rather an accurate representation of the domain.

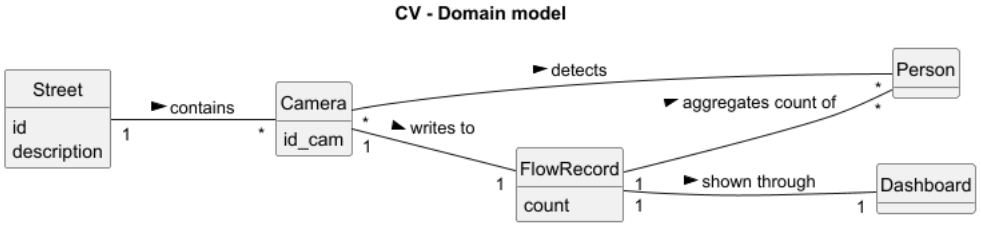

FIGURE 3.1: Domain Model

The system is based on the following main concepts:

- **Zone:** area of urban public space subject to monitoring by the system.
- **Camera:** video capture device installed in the zone, responsible for collecting the image stream.
- **Person:** entity detected by the system across multiple frames.

- **Footfall Record:** periodic record of the number of people present in the zone at a given moment, persisted in the database.
- **Dashboard:** visualisation interface that presents footfall data in real time and the historical pedestrian flow.

3.2 Design Decisions

This section presents the main design decisions made during the conception of the solution, identifying the alternatives considered and justifying the choices made based on the system requirements.

- **Detection Model:** The YOLOv8 family was selected as the basis for people detection, as it represents the state of the art in real-time object detection [12]. Within this family, the YOLOv8n variant, the lightest, was initially adopted, given that the system must operate on hardware without a dedicated GPU, as defined in the non-functional requirements. However, the evaluation in the real environment (Section 4.5) revealed limitations of YOLOv8n, namely false negatives in low-illumination conditions and partial occlusion typical of the monitored environment, which motivated the migration to the YOLOv8s variant. This variant offers greater representational capacity while still remaining operationally viable without a dedicated GPU, as illustrated in Table 3.1. Heavier versions such as YOLOv8m or YOLOv8l, despite being more accurate, do not ensure acceptable performance under the planned hardware conditions.

TABLE 3.1: Comparison between detection model variants

Criterion	YOLOv8n	YOLOv8s	YOLOv8m	YOLOv8l
Speed (CPU)	High	High	Medium	Low
Accuracy (mAP)	Medium	Medium-High	High	Very High
Memory requirements	Low	Low	Medium	High
Compatible without GPU	Yes	Yes	Partially	No
Suitability for use case	Medium	High	Low	Low

- **Tracking Algorithm:** The DeepSORT algorithm was initially considered for tracking people across frames, as analysed in Chapter 2. However, after experimentation, it was found that the instant counting approach using active bounding boxes was sufficient for the use case in question, dispensing with the additional complexity of identity-based tracking, as illustrated in Table 3.2.

TABLE 3.2: Comparison between DeepSORT and active bounding box counting

Criterion	DeepSORT	Active Bounding Boxes
Implementation complexity	High	Low
Computational overhead	High	Negligible
Robustness to occlusion	High	Medium
Instant counting	Indirect	Direct
Cumulative counting	Yes	No
Suitability for use case	Low	High

- **Inference Architecture:** Two architectural approaches were considered for people detection: local inference, running a computer vision model directly on the local machine, and remote inference, using Microsoft's Azure AI Vision service through *Application Programming Interface* (API) calls. Local inference was selected for satisfying the non-functional requirements of real-time performance and supportability without external dependencies, operating at 30 *Frames Per Second* (FPS) without the need for network connectivity. The Azure AI Vision service, despite presenting higher accuracy, as demonstrated in Section 4.4, introduces per-API-call latency incompatible with real-time processing and implies a per-transaction pricing model that is economically disadvantageous for continuous production use. The quantitative evaluation of both approaches is presented in Section 4.4 and the comparison between decision criteria in Table 3.3.

TABLE 3.3: Comparison between local inference and Azure AI Vision

Criterion	Local Inference	Azure AI Vision
Latency	Real time (30 FPS)	High (per API call)
Accuracy (MAE)	2.32	1.48
Operational cost	Fixed (hardware)	Per transaction
Network dependency	None	Required
Data privacy	Full	Data sent externally
Scalability	Limited to hardware	High
Ideal context	POC, real time	Production, high accuracy

- **Database Management System:** The use of a relational database, such as PostgreSQL, and a document database, MongoDB, were considered. Given that the system's data model consists of a single document type with no relationships between entities, a relational database would introduce unnecessary complexity. MongoDB Atlas was selected for offering managed cloud hosting without the need for additional infrastructure, for integrating natively with Python and Power BI through available connectors, and for significantly reducing setup time in a Proof of Concept (POC) context with a constrained deadline.

3.3 Functional and Non-Functional Requirements

The system requirements were gathered based on the project objectives and the needs identified with DevScope. The FURPS+ model, introduced by [21] and extended by [22], was used to structure the non-functional requirements.

3.3.1 Functional Requirements

Functional requirements specify the features that the system must provide to its users. Table 3.4 presents the identified functional requirements.

TABLE 3.4: Functional Requirements

ID	Description
RF01	The system must detect and count people in real time from a camera's video stream.
RF02	The system must periodically record the number of people present in a database.
RF03	The system must provide the current people count in real time.
RF04	The system must allow querying the footfall history by time period.
RF05	The system must identify peak footfall periods.
RF06	The system must provide footfall statistics through an interactive dashboard.

3.3.2 Non-Functional Requirements

Usability

- The dashboard must be intuitive and accessible to users without technical training.
- Footfall information must be presented in a clear and visually comprehensible manner.

Performance

- The system must process the video stream in real time, with acceptable latency for footfall monitoring.
- The system must operate efficiently on hardware without a dedicated GPU.

Reliability

- The system must count people even in situations of partial occlusion.
- The system must automatically recover the connection to the video stream after a temporary network interruption, without the need for manual restart.

Supportability

- The system must be easy to install and configure.
- The system must be easy to maintain and update.

Design Constraints

- The system must be developed in Python.
- The system must be compatible with DevScope's technological infrastructure.

3.4 Solution Design

The solution design is presented according to the 4+1 view model [23], which describes software architecture through five complementary views: logical, implementation, process, physical, and scenarios.

3.4.1 Logical View

The logical view describes the logical structure of the system, identifying the main components and the relationships between them. Figure 3.2 presents an overview of the system.

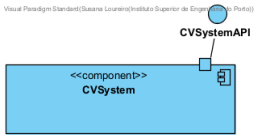

FIGURE 3.2: Logical View (Level 1)

Refining to a second level of granularity, Figure 3.3 details the internal components of the system and the interfaces between them.

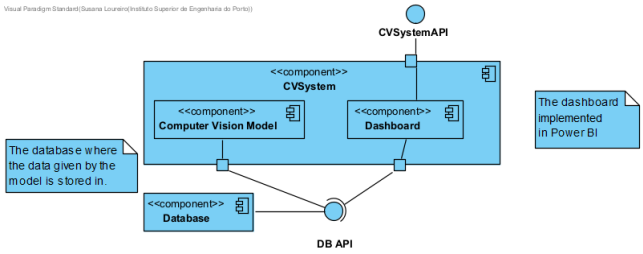

FIGURE 3.3: Logical View (Level 2)

The system is composed of a single top-level component, the `CVSystem`, which encapsulates the solution's processing logic. Internally, this component is organised into two subcomponents: the `Computer Vision Model`, responsible for real-time people detection and counting, and the `Dashboard`, implemented in Power BI, which provides footfall statistics to the user. Both subcomponents communicate with the `Database` through the `DB API` interface, where footfall records are persisted.

3.4.2 Implementation View

The implementation view describes the static organisation of the source code in packages and modules. Figures 3.4 and 3.5 present the two levels of granularity of the implementation view.

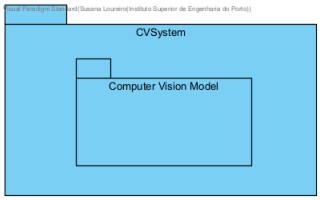

FIGURE 3.4: Implementation View (Level 2)

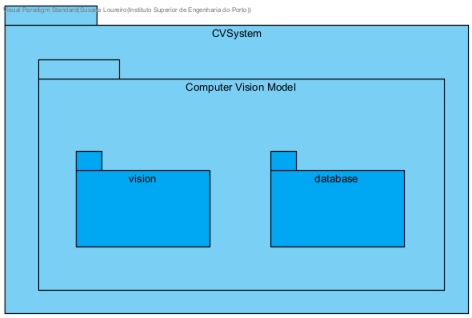

FIGURE 3.5: Implementation View (Level 3)

The system is organised into two main packages within the root `src` package: the `vision` package, responsible for the computer vision pipeline, and the `database` package, responsible for data persistence. Additionally, there are the `settings.json` and `.env` files — the latter for more sensitive information — responsible for the system's global configurations.

3.4.3 Physical View

The physical view describes the mapping of software components onto the hardware infrastructure. Figure 3.6 presents the system's deployment diagram.

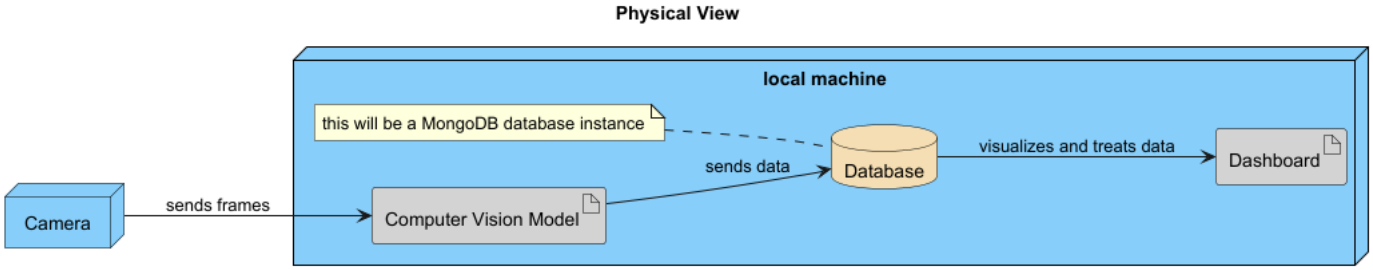

FIGURE 3.6: Physical View

The camera captures the video stream and sends the frames to the local machine, where the computer vision model is executed. The footfall data produced by the model are sent to a MongoDB instance, which serves as the data source for the dashboard developed in Power BI. The database thus constitutes the integration point between the local processing layer and the visualisation layer.

As an architectural alternative, the replacement of the local model with the Azure AI Vision service was considered, as illustrated in Figure 3.7. In this approach, frames captured by the camera are sent to the Azure AI Vision API, which returns the detections to the local machine. The database layer and dashboard remain unchanged. This alternative was discarded for the reasons described in Section 3.2.

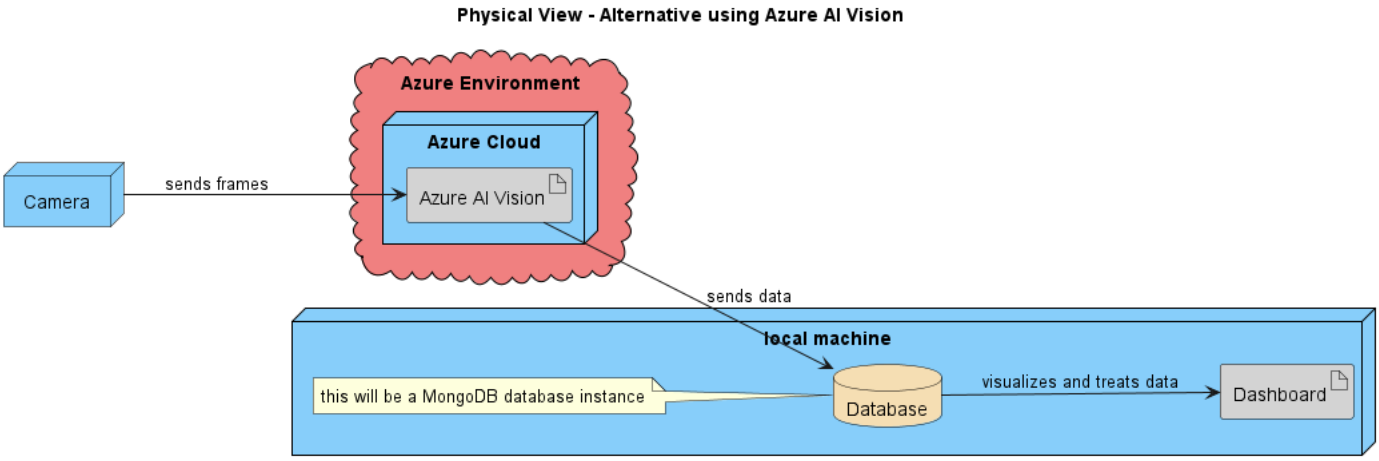

FIGURE 3.7: Physical View – Alternative with Azure AI Vision

3.4.4 Process View

The process view describes the dynamic flow of the system, illustrating the interactions between components during processing. The processes are detailed in Section 3.5, where each use case is accompanied by its respective sequence diagram.

3.5 Use Cases

The use cases were defined based on the functional requirements identified in Section 3.3. Figure 3.8 presents the system's use case diagram.

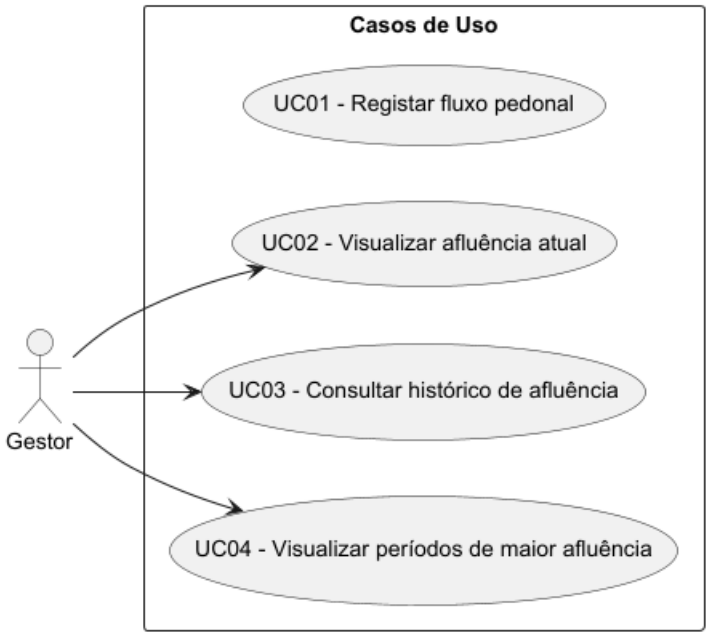

FIGURE 3.8: Use Case Diagram

3.5.1 UC01: Record Pedestrian Flow

This use case describes the automatic and continuous process of detecting and recording people footfall. As illustrated in Figure 3.9, while the camera is active, each captured frame is sent to `Main`, which orchestrates the pipeline: the `Detector` identifies people and returns their respective bounding boxes, and the `Counter` tallies the bounding boxes currently active. Every ten minutes, the count is persisted in the database by the `DatabaseHandler`, together with the camera identifier and the timestamp.

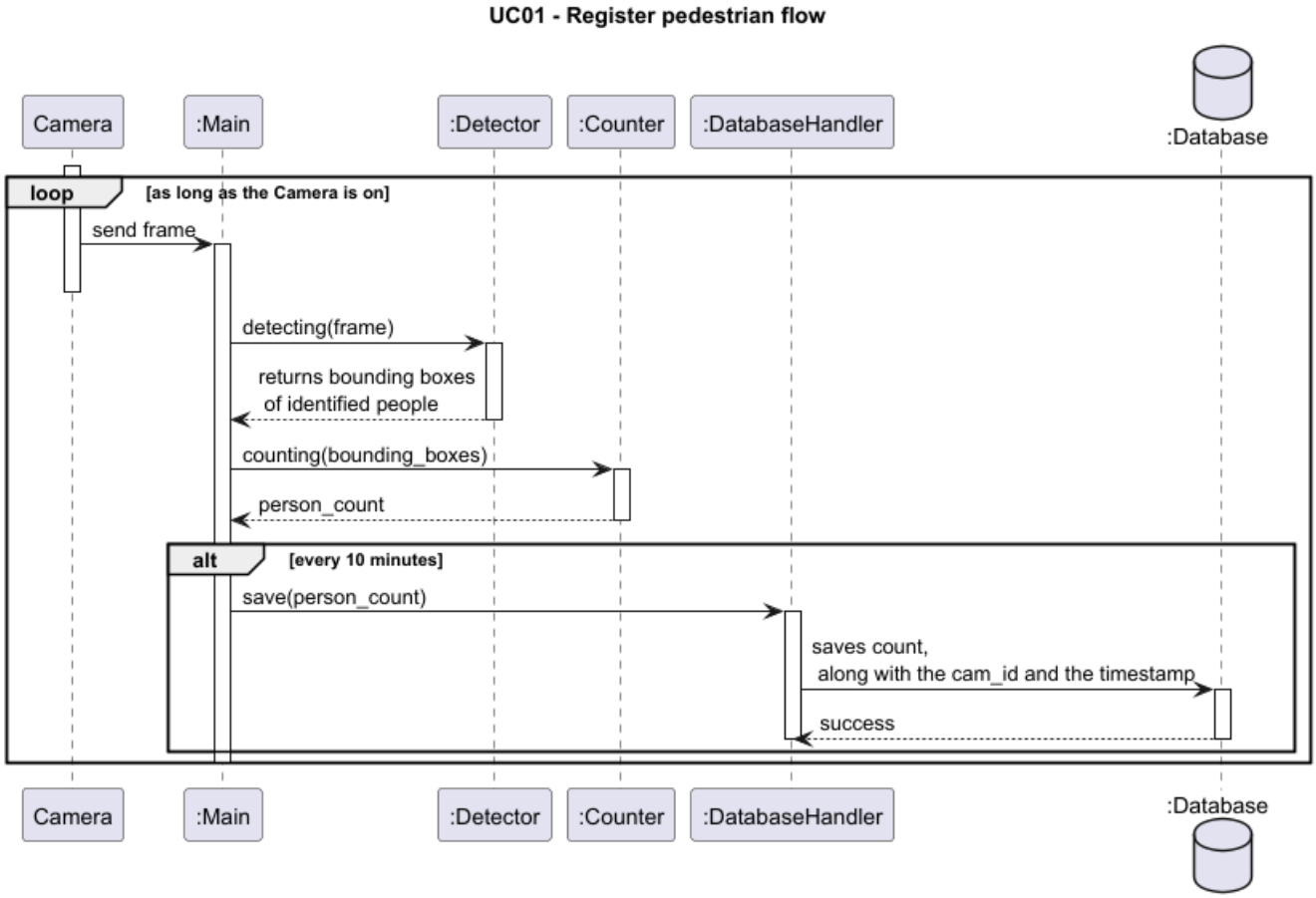

FIGURE 3.9: UC01 Diagram

3.5.2 UC02: View Current Footfall

This use case describes the process by which the manager consults the current footfall in the monitored zone. As illustrated in Figure 3.10, the manager accesses the dashboard and requests the current count view, being asked which camera is intended. After selection, the dashboard queries the database for the most recent count and presents the information to the manager.

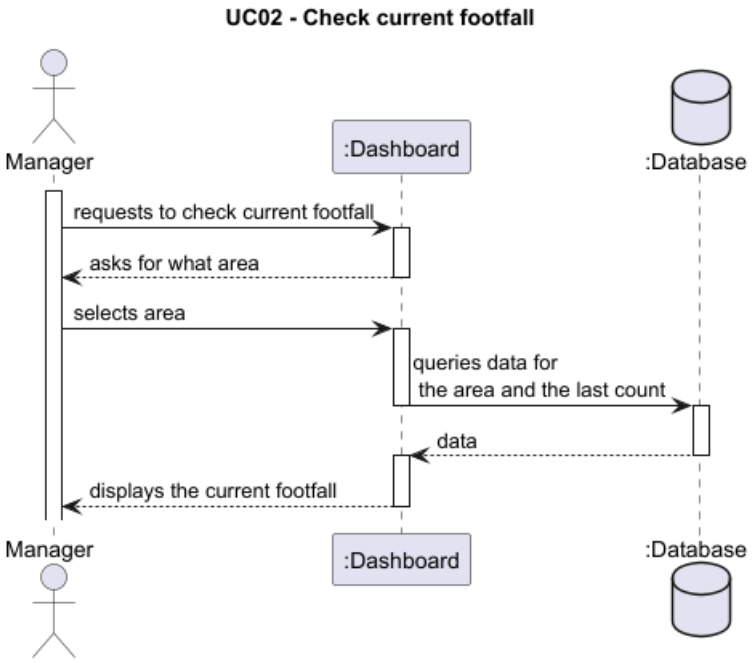

FIGURE 3.10: UC02 Diagram

3.5.3 UC03: Query Footfall History

This use case describes the process by which the manager consults the footfall history of the monitored zone. As illustrated in Figure 3.11, the manager requests the footfall history from the dashboard, the dashboard queries the database and presents the historical data.

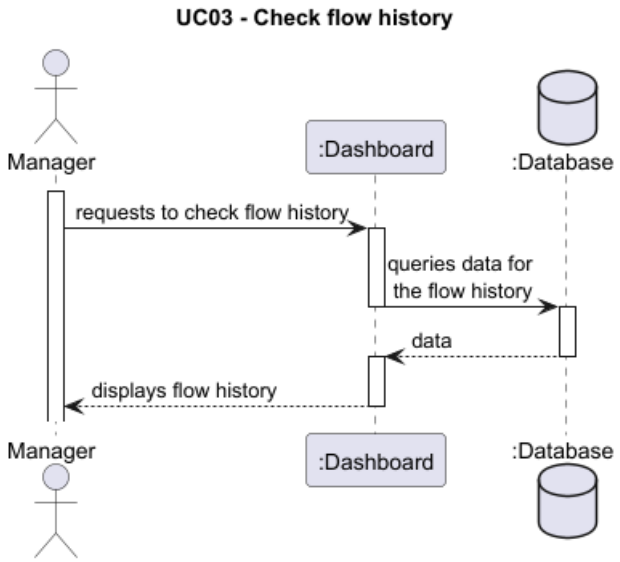

FIGURE 3.11: UC03 Diagram

3.5.4 UC04: View Peak Footfall Periods

This use case describes the process by which the manager identifies peak footfall periods based on historical data. As illustrated in Figure 3.12, the manager requests peak footfall periods from the dashboard, the dashboard queries the database and presents the periods with the highest recorded number of people.

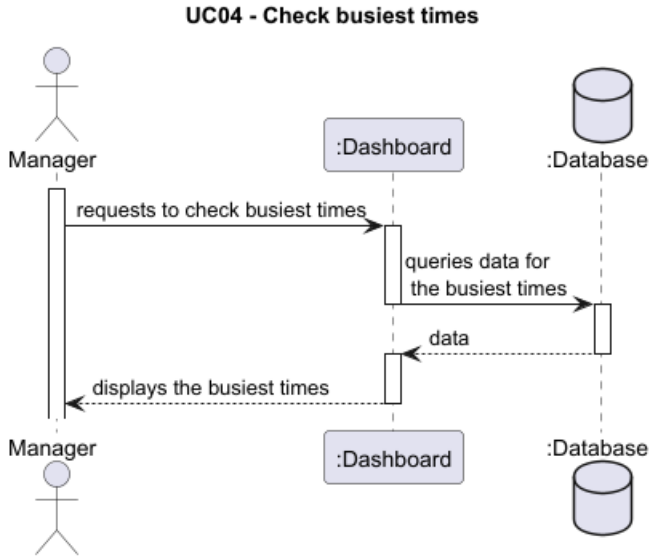

FIGURE 3.12: UC04 Diagram

Chapter 4

Solution Implementation and Experimentation

This chapter describes the implementation of the solution proposed in the previous chapter. The technical details of the computer vision pipeline are presented, along with the technologies used, the model training data, the evaluation approach, and the results obtained.

4.1 Implementation Description

The solution was developed in Python and is organised into two main packages within the `src` directory: the `vision` package, responsible for the computer vision pipeline, and the `database` package, responsible for data persistence. The system's global configurations are managed through the `settings.json` file. There is also a `.env` file containing more sensitive configurations, such as the database connection string and the IP address for connecting to the camera stream. The `main` module orchestrates the execution of all components, coordinating the data flow between the vision pipeline and the persistence layer.

The solution's architecture is organised into three layers: *model*, *database*, and *dashboard*. Frames captured by the camera are processed by the computer vision model, the resulting count is persisted in the MongoDB database, provisioned in the cloud using MongoDB Atlas, and the data are subsequently visualised through the dashboard implemented in Power BI.

4.1.1 Computer Vision Pipeline

The `vision` package implements the people detection pipeline, composed of the following components:

- **Detector**: uses the YOLOv8s model, as justified in Section 3.2, to identify people in each frame and returns their respective bounding boxes;
- **Counter**: tallies the active bounding boxes at each moment, producing the instantaneous count of people present.

The video stream is consumed through a dedicated thread, implemented in the `StreamReader` class, which continuously reads frames from the camera in the background. This approach is necessary to ensure the stability of stream reading: without it, the OpenCV buffer is not emptied frequently enough, resulting in unexpected program termination. Database persistence is equally executed in a separate thread, preventing write operations from blocking the main frame processing loop.

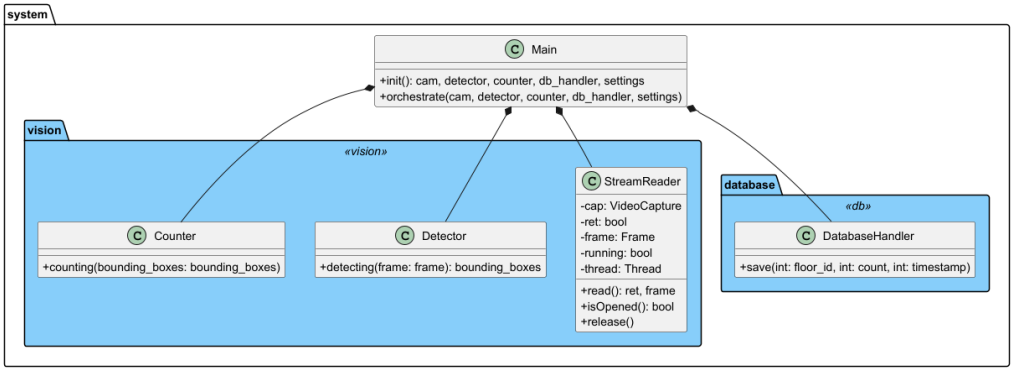

FIGURE 4.1: Solution module diagram

Images captured by the camera are neither persisted nor stored in the system — only the people count is saved in the database, ensuring the protection of individuals' identities. Additionally, a pre-trained YOLOv8n model is applied for face detection, automatically anonymising visible faces in the video stream, ensuring the privacy of individuals even in the event of unauthorised access to the camera.

4.1.2 Capture Environment Setup

The capture environment was configured at one of the windows of DevScope's premises. The capture device consists of a Raspberry Pi with an attached Camera Module, compact in size, positioned to maximise the visual coverage area over the monitored public street.

The Raspberry Pi acts as a stream server, transmitting the video stream over a wireless network using the HTTP Live Streaming (HLS) protocol, consumed by OpenCV via the `playlist.m3u8` address configured in the application's `.env` file. In an initial phase, the program periodically terminated due to timeouts when reading the stream. The issue was resolved through adjustments to the program serving as the Raspberry Pi's HTTP server.

The stream was configured with a resolution of 480p at 5 frames per second. This architecture dispenses with any physical connection between the capture device and the machine where the computer vision model is executed.

4.1.3 Database

The `database` package is responsible for the persistence of footfall data. As justified in Section 3.2, MongoDB Atlas was adopted as the database management system.

Each footfall record is stored as a document with the following structure:

- `cam_id`: identifier of the camera in the monitored zone;
- `count`: number of people counted at that moment;
- `timestamp`: record moment in Unix format;
- `time`: record moment in date and time format.

An example of a document stored in the database is presented below:

```
{
    "cam_id": 1,
    "count": 5,
```

```
    "timestamp": 1777288530.2494867,
    "time": "2026-04-27T11:15:30.249487+00:00"
}
```

LISTING 4.1: Example of a footfall record document in MongoDB

Every ten minutes, the `DatabaseHandler` persists the current count in the database, together with the camera identifier and the temporal fields.

4.1.4 Dashboard

The dashboard was developed in Microsoft Power BI, a technology adopted by DevScope in its internal data ecosystem, which facilitated integration and reduced the learning curve. Power BI connects directly to the MongoDB instance and enables visualisation of footfall data in real time and from a historical perspective.

It is worth noting that, as this is a POC, the dashboard requires manual refresh to reflect the most recent data, given that Power BI does not natively support streaming from MongoDB without additional infrastructure. This limitation is acknowledged and discussed in Section 5.2.

The dashboard provides the following visualisations:

- Current footfall in the monitored zone;
- Footfall history over time;
- Identification of peak footfall periods.

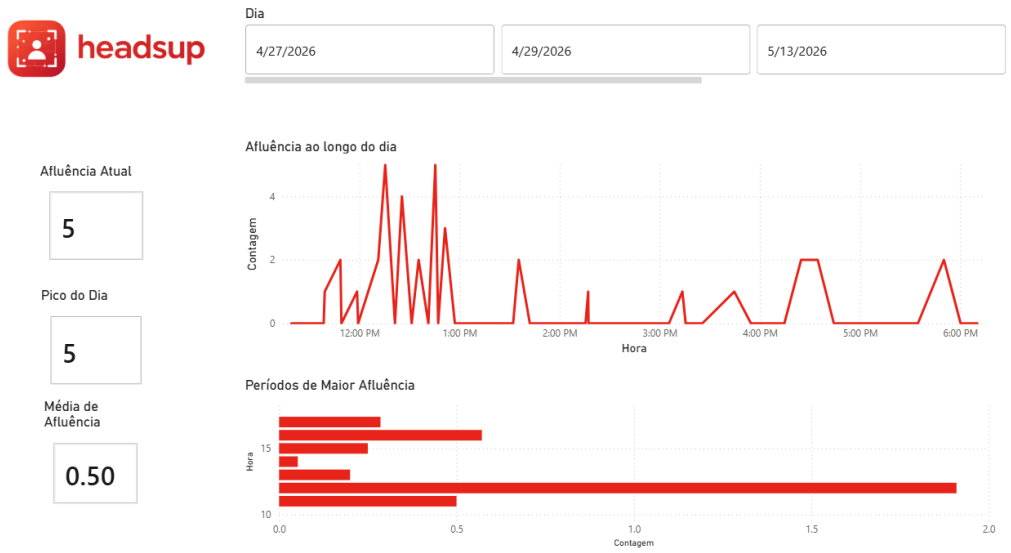

FIGURE 4.2: Footfall monitoring dashboard in Power BI

4.2 Datasets and Model Training

The selection of datasets used in fine-tuning resulted from an iterative process. In an exploratory phase, two generic pedestrian detection datasets were used (the *Pedestrian Detection* [24] and the *Pedestrian/Human Detection* [25]) applied to the YOLOv8n variant.

mAP is calculated as the mean of the Average Precision (AP) values obtained for each class, integrating the precision-recall curve: the closer to 1, the better the model detects objects correctly without false positives or negatives.

Despite the high mAP values obtained in this training, evaluation in the real environment revealed that the model did not outperform the base model without fine-tuning. Two main causes were identified: the composition of the datasets, mostly consisting of medium-to-high pedestrian density scenes without sufficient examples of low footfall, and the absence of images representative of the perspective and illumination conditions of the monitored environment. This approach was abandoned and the process was redirected, as described below.

The final fine-tuning set is composed of four datasets, retaining the two previous ones and adding two urban surveillance datasets with greater diversity of contexts and pedestrian densities: *MOT17* [26], available on the Kaggle platform, composed of street video sequences with bounding box annotations and visibility levels per pedestrian; and *CityPersons* [27], available on the Roboflow Universe platform, composed of 3,475 images of pedestrians in an urban environment. These two datasets were added precisely because they represent real urban surveillance sequences with variable pedestrian density, including low-footfall scenes, thus addressing the main gap identified in the exploratory phase. Simultaneously, the base variant was migrated from YOLOv8n to YOLOv8s, motivated by the limitations observed in the real environment described in Section 4.5.

MOT17 required conversion from its native annotation format to the YOLO format. The annotations were stored in `gt/gt.txt` files per sequence, in the format *frame, id, x, y, w, h, conf, class, visibility*. Only annotations corresponding to the pedestrian class ($class = 1$) with visibility greater than 0.3 were retained, in order to exclude instances with severe occlusion that could introduce noise into the training. Since each MOT17 sequence exists in three variants with different detectors (DPM, FRCNN, SDP) sharing the same images, only the SDP variant was used to avoid triplication of the dataset. The *Pedestrian/Human Detection* also required conversion from COCO JSON to YOLO format, filtering to the *person* class. CityPersons was filtered to the *ped* class, discarding the *ignore* and *None* classes present in the original annotations.

After merging the four datasets into a unified set, the images were randomly split in a 75/15/10 ratio between training, validation, and test sets, respectively, with a fixed seed for reproducibility.

Training was run for 50 epochs with an image size of 640px and a batch size of 16 samples per iteration, on a T4 GPU instance available on the Kaggle platform, as the local hardware does not have a dedicated GPU. The resulting trained model was exported and integrated into the solution's computer vision pipeline. The training results are presented in Table 4.1.

TABLE 4.1: YOLOv8s model training results

Metric	**Value**
mAP@0.5	0.937
mAP@0.5:0.95	0.724
Precision	0.954
Recall	0.879

The trained model achieved a mAP@0.5 of 0.937 and a precision of 0.954, values substantially superior to those of the vanilla YOLOv8s model evaluated on the same validation set

(mAP@0.5 = 0.674, precision = 0.771), confirming that fine-tuning with the four datasets produced a significant improvement in people detection capability in an urban surveillance context. However, high training metrics do not guarantee equivalent performance in the real environment. The comparative evaluation presented in Section 4.4 will allow an assessment of the extent to which these gains translate into more accurate counts under the specific conditions of the monitored space.

An alternative or complementary approach to fine-tuning would be the collection of images directly in the production environment, or the application of data augmentation techniques that simulate the specific camera and illumination conditions of the monitored space. These directions were identified as future work in Section 5.2.

4.3 Testing

The testing strategy adopted was organised into two complementary dimensions: verification, confirming that the system was built correctly, and validation, confirming that the system produces correct results in relation to the real problem. The validation results are explained in detail in Section 4.4.

4.3.1 Verification

Functional verification of the system was carried out incrementally throughout development. In an initial phase, publicly available videos from YouTube and Shutterstock were used to confirm the correct functioning of the detection and tracking pipeline under controlled conditions, before moving to tests with real images. This approach made it possible to isolate and correct pipeline problems without depending on physical infrastructure.

TABLE 4.2: Functional verification test cases

Requirement	Test Case	Expected Result	Result
RF01	People detection in public video	*Bounding boxes* visible in each *frame*	Pass
RF01	Detection with camera used in the solution	*Bounding boxes* in real environment	Pass
RF02	Persistence after 10 minutes	Document created in MongoDB Atlas	Pass
RF02	Structure of the persisted document	Fields `cam_id`, `count`, `timestamp`, `time` present	Pass
RF03	Current count in the dashboard	Value updated after *refresh*	Pass
RF04	History query by period	Data filtered correctly in Power BI	Pass
RF05	Identification of footfall peaks	Peak periods visible in the *dashboard*	Pass
RF06	Facial anonymisation	Faces not visible in the video *stream*	Pass

Data persistence was verified by confirming the records written to the MongoDB Atlas instance, validating the correct structure of the documents and the integrity of the `count` and `timestamp` fields. The connection between the database and the dashboard was verified through the visualisation of real occupancy data in Power BI, as illustrated in Figure 4.2.

Subsequently, tests were conducted with the camera used in the final company solution, capturing images from the real environment, which allowed the model's inference parameters to be adjusted, namely the confidence threshold (`conf=0.3`) and the IoU threshold (`iou=0.4`), in order to reduce spurious detections and improve detection stability.

4.3.2 Validation

Quantitative validation of the system was carried out through a comparative evaluation described in detail in Section 4.4. 50 frames were uniformly extracted from outdoor environment videos, for which a ground truth was produced by manual counting. The comparison between the counts produced by the system and the ground truth constitutes the primary validation evidence for the solution, quantified using the Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and exact match rate metrics.

4.4 Model Evaluation and Comparison

The quantitative evaluation of the solution was carried out using five publicly available urban public space videos from YouTube, representing different contexts and footfall levels: a busy commercial street, a pedestrian zone near a metro station, a low-footfall urban street, a leisure area by the river, and a tree-lined promenade with moderate pedestrian flow. For each video, 50 frames were uniformly extracted based on the interval calculated as `total_frames // 50`. For each frame, a manual count of the number of visible people was performed, constituting the ground truth. It is worth noting that, in videos with people in motion or partial occlusion, manual counting may not be trivial, introducing some inherent uncertainty in the ground truth itself.

The seven approaches used for comparison were:

- YOLOv8n model pre-trained on the COCO dataset, without any adaptation, executed with default parameters (vanilla);
- YOLOv8n model with experimentally tuned inference parameters (`conf=0.6`, `iou=0.7`, `classes=0`);
- YOLOv8n model subjected to fine-tuning with two generic pedestrian detection datasets, executed with default parameters;
- YOLOv8s model pre-trained on the COCO dataset, without any adaptation, executed with default parameters (vanilla);
- YOLOv8s model with experimentally tuned inference parameters (`conf=0.3`, `iou=0.4`, `classes=0`);
- YOLOv8s model subjected to fine-tuning with the four datasets described in Section 4.2, executed with default parameters;
- Microsoft's Azure AI Vision service, made available through an API.

To ensure the consistency of comparisons, all approaches were integrated into the same pipeline and applied to the same videos. The count recorded in each evaluation frame was compared with the corresponding ground truth.

Given that the system's objective is to produce an accurate count of the number of people present, the metrics adopted for quantitative evaluation were MAE, MAPE, and the exact

match rate, i.e., the percentage of frames in which the model's count exactly matched the ground truth. These metrics are more appropriate than precision, recall, and F1-score for counting systems, as they directly quantify the deviation between the estimate and the actual value, rather than treating the problem as a binary classification.

TABLE 4.3: Aggregated results of the comparative model evaluation (average across five videos)

Metric	v8n vanilla	v8n tuned	v8n trained	v8s vanilla	v8s tuned	v8s trained	Azure
MAE	4.85	2.28	3.13	9.16	2.31	3.15	1.48
MAPE (%)	80.24	36.45	69.06	186.34	39.03	61.34	26.60
Exact (%)	6.00	18.80	10.40	2.40	24.40	11.60	27.60

The aggregated results, presented in Table 4.3, reveal a consistent pattern across the two model families tested: in both, inference parameter tuning constitutes the most effective intervention, and fine-tuning does not outperform the tuned model in the real environment. Among the YOLOv8n-based approaches, parameter tuning reduced the MAE from 4.85 to 2.28 and the MAPE from 80.24% to 36.45% compared to the vanilla model. The pattern repeats with YOLOv8s: the tuned model achieved a MAE of 2.31 and a MAPE of 39.03%, clearly outperforming the vanilla (MAE = 9.16, MAPE = 186.34%) and the trained model (MAE = 3.15, MAPE = 61.34%).

The comparison between variants reveals that the vanilla YOLOv8s performs worse than vanilla YOLOv8n (MAE 9.16 vs 4.85), suggesting that the greater capacity of the s model, without parameter tuning, tends to produce more detections in the evaluated videos. However, after tuning, YOLOv8s marginally outperforms YOLOv8n (MAE 2.31 vs 2.28, exact match rate 24.40% vs 18.80%), with the most expressive advantage in the exact match rate, which reflects greater frame-to-frame counting stability. The YOLOv8n model subjected to fine-tuning, despite the high mAP values obtained during training, as presented in Section 4.2, did not demonstrate a consistent improvement over the tuned model in the real environment, recording a MAE of 3.13 and a MAPE of 69.06%. The same pattern is observed in the trained YOLOv8s (MAE = 3.15, MAPE = 61.34%), suggesting that the datasets used, although of an urban outdoor context, do not sufficiently represent the specific conditions of the evaluated videos, particularly in low-footfall scenes where both trained models tend to produce spurious detections.

Across all local approaches, the low exact match rate reflects the inherent difficulty of accurate counting in scenes with partial occlusion, orientation variations, and multiple simultaneous people. This metric is particularly demanding in counting systems, since a discrepancy of just one person is sufficient for a frame not to be counted as an exact match. MAE therefore constitutes the most representative indicator of estimate quality for the use case in question.

The Azure AI Vision service presented the best results across all metrics, with a MAE of 1.48, MAPE of 26.60%, and an exact match rate of 27.60%, outperforming all local approaches. This superiority is explained by the fact that the service uses large-scale models trained on substantially larger and more diverse data volumes, without the hardware limitations that conditioned local choices. However, its adoption as the final solution was discarded for the reasons described in Section 3.2.

In light of the results obtained, the approach adopted in the final solution consists of the YOLOv8s model with tuned inference parameters (`conf=0.3`, `iou=0.4`), having demonstrated the best aggregated performance among the local approaches evaluated, with a MAE of 2.31

and an exact match rate of 24.40%, outperforming all other local approaches across both metrics.

FIGURE 4.3: Examples of people detection with the tuned YOLOv8s model

4.5 Real-World Evaluation

In addition to the evaluation carried out with YouTube videos in Section 4.4, a real-world evaluation was conducted using video images captured directly by the camera installed at DevScope's premises, described in Section 4.1.1. The aim of this evaluation is to verify the solution's performance under the specific perspective, illumination, and pedestrian density conditions of the production monitoring environment, which the YouTube videos do not faithfully replicate.

Four video segments were captured throughout the day, representing distinct illumination and footfall conditions: a morning period with overcast sky and low footfall, and three afternoon periods captured at the beginning, middle, and end of the afternoon, with direct sunlight and moderate to high footfall. For each segment, 50 frames were uniformly extracted, for a total of 200 manually annotated frames, following the methodology described in Section 4.4.

The approaches evaluated in this context were the YOLOv8s family variants, as these are the relevant ones for the final solution: the vanilla model, the model with tuned parameters (`conf=0.3`, `iou=0.4`), and the model subjected to fine-tuning with the four datasets described in Section 4.2. The Azure AI Vision service was excluded from this evaluation as it was discarded as the final solution for the reasons described in Section 3.2, regardless of its quantitative performance.

TABLE 4.4: Real-world evaluation results (average across four segments)

Metric	YOLOv8s vanilla	YOLOv8s tuned	YOLOv8s trained
MAE	1.71	2.10	2.74
MAPE (%)	72.54	70.52	93.38
Exact match rate (%)	20.00	22.00	12.50

The real-world results, presented in Table 4.4, reveal a different behaviour from that observed in the YouTube video evaluation. The vanilla model achieved the best MAE (1.71),

outperforming the tuned model (2.10) and the trained model (2.74). This result contrasts with those obtained in Section 4.4 and is explained by the nature of the monitored environment: the zone presents low pedestrian density, with few frames containing more than two or three people simultaneously visible. In this context, the reduced confidence threshold of the tuned model (`conf=0.3`) introduces spurious detections that penalise the MAE, while the vanilla model, with its more restrictive default parameters, tends to be more conservative and accurate in low-footfall scenes.

Segment-by-segment analysis reveals that the tuned model achieved competitive results in the morning and higher-movement afternoon segments, but degraded significantly in one of the afternoon segments (MAE = 3.78), suggesting sensitivity to specific illumination and sun angle conditions. It was also found that partial occlusion caused by elements of the urban scene — namely the tree present in the camera's field of view — contributes to false negatives in certain frames, as visible in Figure 4.4. This limitation is discussed in greater detail in Section 5.2. The trained model maintained the pattern observed in the public video evaluation, consistently presenting the worst performance of the three, with an average MAPE of 93.38%, confirming the inadequacy of the training datasets to the conditions of the monitored environment.

The exact match rate of the tuned model (22.00%) is slightly higher than that of the vanilla (20.00%), suggesting that despite the higher MAE, the tuned model more frequently achieves the exact count. Given the set of results obtained, the tuned YOLOv8s model remains the approach adopted in the final solution, for presenting the best balance between performance in variable-density scenarios — since it clearly outperforms the vanilla as demonstrated in Section 4.4 — and stability in the real environment.

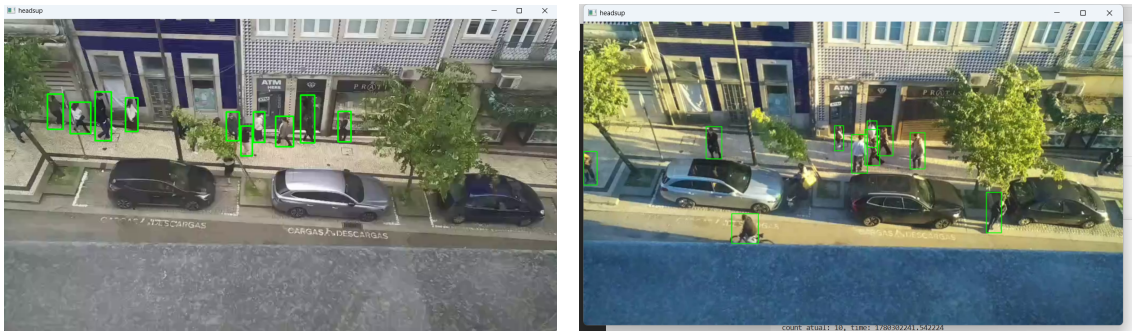

FIGURE 4.4: Examples of people detection with the tuned YOLOv8s model in production

Chapter 5

Conclusions

This chapter presents a synthesis of the results obtained throughout the project, evaluates the degree to which the defined objectives were achieved, identifies the limitations of the developed solution, and proposes directions for future work.

5.1 Achieved Objectives

The main objective of the project was the development of a pedestrian footfall monitoring system for public space, capable of identifying and counting people in real time through computer vision, producing a historical footfall record, and identifying peak periods of people concentration. Table 5.1 presents the degree of achievement of each objective.

TABLE 5.1: Degree of objective achievement

Objective	**Status**	**Notes**
Detect and count people in real time from a camera's video stream.	Completed	Implemented with the YOLOv8s model with tuned parameters, as described in Section 4.1.1.
Periodically record the number of people present in a database.	Completed	The `DatabaseHandler` persists the count every ten minutes in MongoDB Atlas, as described in Section 4.1.3.
Produce a historical footfall record that allows the extraction of relevant statistics.	Completed	Historical data are visualised through the Power BI dashboard, as described in Section 4.1.4.
Identify peak footfall periods.	Completed	The dashboard provides a dedicated visualisation of peak people concentration periods, as illustrated in Figure 4.2.
Ensure the privacy of monitored individuals.	Completed	Images are not persisted and faces are automatically anonymised by a YOLOv8n face detection model, as described in Section 4.1.1.

Table 5.1 – continued

Objective	Status	Notes
Evaluate and compare different detection approaches.	Completed	Seven approaches were compared, including YOLOv8n and YOLOv8s variants with and without fine-tuning and the Azure AI Vision service, as described in Section 4.4.

were achieved. The approach adopted in the final solution, the YOLOv8s model with tuned inference parameters (`conf=0.3`, `iou=0.4`), demonstrated the best aggregated performance amo All defined objectivesng the local approaches evaluated on public videos, with a MAE of 2.31 and an exact match rate of 24.40%.

The real-world evaluation confirmed the viability of the solution under the specific conditions of the monitored space, with a MAE of 2.10 and an exact match rate of 22.00% for the tuned model. It was found that in low pedestrian density scenarios, characteristic of the monitored environment, the vanilla model achieved a slightly lower MAE (1.71), suggesting that the reduced confidence threshold introduces some spurious detections in these conditions. This observation constitutes a concrete direction for future refinement of inference parameters in production.

5.2 Limitations and Future Work

Despite the achieved objectives, the developed solution presents a set of limitations that are important to acknowledge, and which simultaneously constitute opportunities for future development.

The absence of a dedicated GPU on the local hardware significantly conditioned the design decisions, namely the choice of the YOLOv8s variant and the constraints in the fine-tuning process, including the number of epochs and the size of the datasets used. The availability of hardware with a dedicated GPU would allow exploring more accurate model variants, such as YOLOv8m or YOLOv8l, as well as more extensive training processes with larger datasets, potentially improving real-world performance.

The transmission of the video stream over the corporate network, requiring a *Virtual Private Network* (VPN), introduces latency and instability in the quality of the stream consumed by the model, conditioning the sharpness of the processed frames. This limitation could not be circumvented given that the Raspberry Pi does not support transmission via *Real Time Streaming Protocol* (RTSP), a protocol that would offer lower latency and greater stability. Replacing the capture device with more capable hardware would constitute a relevant direction for future work to mitigate this issue.

The fine-tuning performed revealed a partial inadequacy between the training data and the production environment, as the datasets used, although of an urban outdoor context, do not sufficiently represent the specific perspective, camera, and illumination conditions of the monitored space. The most promising direction to address this limitation would be the collection of images directly in production to build a dataset specific to the monitored space, complemented by the application of data augmentation techniques that simulate the variable illumination conditions observed throughout the day. A model trained under these conditions

would significantly reduce both spurious detections in low-footfall scenes and false negatives under adverse illumination conditions.

Partial occlusion also constitutes a relevant limitation of the system. In urban contexts with vegetation or other obstacles in the camera's field of view, as illustrated in Figure 4.4, people who are partially obstructed tend not to be detected by the model, contributing to the false negatives observed in the real-world evaluation. Mitigating this limitation would require the collection of occluded images directly in production for inclusion in the training set, as well as exploring camera installation angles that minimise the interposition of obstacles in the field of view.

The dashboard developed in Power BI requires manual refresh to reflect the most recent data, given that the native integration between Power BI and MongoDB does not support real-time streaming without additional infrastructure. A future direction would involve implementing an intermediate layer, for example, through the Power BI Streaming API or an Apache Kafka-based solution, that would allow automatic dashboard updates.

Finally, the current solution monitors a single zone through a single camera. Extending to multiple cameras and zones, with aggregation of footfall data in a unified dashboard, would constitute a natural evolution of the modular architecture adopted, enabling the solution to scale to more complex urban contexts.

5.3 Final Remarks

The curricular internship at DevScope made it possible to develop a functional pedestrian footfall monitoring system for public space, integrating computer vision, cloud data persistence, and dashboard visualisation. The project provided an experience close to the real software development context, from requirements definition through to the quantitative evaluation of the solution in production.

The comparison between seven detection approaches revealed that the best local solution was not necessarily the most complex: the careful tuning of inference parameters demonstrated a greater impact than fine-tuning with generic datasets, underlining that the quality and representativeness of the training data have a more decisive impact on real-world performance than model complexity. This is, arguably, the most relevant technical takeaway from the project.

The project also revealed a less technical, but equally formative dimension: adapting to constraints imposed by the organisational context. The initial scope foresaw monitoring people inside DevScope's premises, by floor. However, the absence of authorisation for image capture in an internal environment required the project to be redirected towards monitoring outdoor public space. This change had a direct impact on subsequent technical decisions, namely the camera perspective, variable illumination conditions, and training dataset composition, and illustrates a reality common in professional software development: requirements change, often for reasons outside the technical team's control, and the ability to adapt the solution without losing the thread of the project is just as important as technical competence itself.

From a personal standpoint, this project represented a first in-depth encounter with the development of computer vision systems applied to a real problem. The challenges encountered, from hardware constraints to privacy considerations to corporate network instability, required making design decisions under uncertainty and with limited resources, a skill I consider more valuable than any quantitative result obtained. With more time and resources, I would have

invested in collecting data directly in production from the start of the project, which I believe would be the change with the greatest impact on the solution's final performance.

Bibliography

[1] D. A. Forsyth and J. Ponce, *Computer Vision: A Modern Approach*, 2nd ed. Prentice Hall, 2012, ISBN: 978-0-13-608592-8. [Online]. Available: https://eclass.hmu.gr/modules/document/file.php/TM152/Books/Computer%20Vision%20-%20A%20Modern%20Approach%20-%20D.%20Forsyth%2C%20J.%20Ponce.pdf.

[2] S. V. Vasantha, B. Kiranmai, M. A. Hussain, S. S. Hashmi, L. Nelson, and S. Hariharan, "Face and object detection algorithms for people counting applications," *2nd International Conference on Automation, Computing and Renewable Systems, ICACRS 2023 - Proceedings*, pp. 1188–1193, 2023. DOI: 10.1109/ICACRS58579.2023.10405114. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10405114.

[3] G. Curiel, K. Guerrero, D. Gómez, and D. Charris, "A computer vision based system for human detection and automatic people counting," *Transactions on Energy Systems and Engineering Applications*, vol. 5, pp. 1–14, 2 Oct. 2024, ISSN: 2745-0120. DOI: 10.32397/tesea.vol5.n2.624. [Online]. Available: https://revistas.utb.edu.co/tesea/article/view/624.

[4] D. Ballard, Y. Lecun, B. Boser, *et al.*, "Backpropagation applied to handwritten zip code recognition," [Online]. Available: http://direct.mit.edu/neco/article-pdf/1/4/541/811941/neco.1989.1.4.541.pdf.

[5] H. Mokayed, T. Z. Quan, L. Alkhaled, and V. Sivakumar, "Real-time human detection and counting system using deep learning computer vision techniques," *Artificial Intelligence and Applications*, vol. 1, pp. 205–213, 4 Oct. 2023, ISSN: 28110854. DOI: 10.47852/bonviewAIA2202391. [Online]. Available: https://ojs.bonviewpress.com/index.php/AIA/article/view/391.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[7] E. P. Myint and M. M. Sein, "People detecting and counting system," *LifeTech 2021 - 2021 IEEE 3rd Global Conference on Life Sciences and Technologies*, pp. 289–290, Mar. 2021. DOI: 10.1109/LifeTech52111.2021.9391951. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9391951.

[8] I. J. C. Valencia, E. P. Dadios, A. M. Fillone, J. C. V. Puno, R. G. Baldovino, and R. K. C. Billones, "Vision-based crowd counting and social distancing monitoring using tiny-yolov4 and deepsort," *2021 IEEE International Smart Cities Conference, ISC2 2021*, Sep. 2021. DOI: 10.1109/ISC253183.2021.9562868. [Online]. Available: https://ieeexplore.ieee.org/document/9562868.

[9] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.

[10] M. C. Le, M. H. Le, and M. T. Duong, "Vision-based people counting for attendance monitoring system," *Proceedings of 2020 5th International Conference on Green Technology and Sustainable Development, GTSD 2020*, pp. 349–352, Nov. 2020. DOI: 10.1109/GTSD50082.2020.9303117. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9303117.

[11] A. K. Singh, D. Singh, and M. Goyal, "People counting system using python," *Proceedings - 5th International Conference on Computing Methodologies and Communication, ICCMC 2021*, pp. 1750–1754, Apr. 2021. DOI: 10.1109/ICCMC51019.2021.9418290. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9418290/citations#citations.

[12] T. A. R. Shyaa and A. A. Hashim, "Enhancing real human detection and people counting using yolov8," *BIO Web of Conferences*, vol. 97, p. 00061, Apr. 2024, ISSN: 21174458. DOI: 10.1051/bioconf/20249700061. [Online]. Available: https://www.bio-conferences.org/articles/bioconf/abs/2024/16/bioconf_iscku2024_00061/bioconf_iscku2024_00061.html.

[13] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR 2021 - 9th International Conference on Learning Representations*, Oct. 2020. [Online]. Available: https://arxiv.org/pdf/2010.11929.

[14] N. Krishnachaithanya, G. Singh, S. Sharma, *et al.*, "People counting in public spaces using deep learning-based object detection and tracking techniques," *Proceedings of International Conference on Computational Intelligence and Sustainable Engineering Solution, CISES 2023*, pp. 784–788, 2023. DOI: 10.1109/CISES58720.2023.10183503. [Online]. Available: https://ieeexplore.ieee.org/document/10183503.

[15] M. Razzok, A. Badri, I. E. Mourabit, Y. Ruichek, and A. Sahel, "Pedestrian detection and tracking system based on deep-sort, yolov5, and new data association metrics," *Information 2023, Vol. 14, Page 218*, vol. 14, p. 218, 4 Apr. 2023, ISSN: 20782489. DOI: 10.3390/info14040218. [Online]. Available: https://www.mdpi.com/2078-2489/14/4/218/htm%20https://www.mdpi.com/2078-2489/14/4/218.

[16] M. Desai, H. Mewada, I. M. Pires, and S. Roy, "Evaluating the performance of the yolo object detection framework on coco dataset and real-world scenarios," *Procedia Computer Science*, vol. 251, pp. 157–163, 3 Jan. 2024, ISSN: 18770509. DOI: 10.1016/j.procs.2024.11.096. [Online]. Available: https://doi.org/10.1109/jproc.2023.3238524.

[17] T. Fu, Y. Chen, Z. Chen, M. Zhao, B. Li, and X. Xue, "Crowdtrack: A benchmark for difficult multiple pedestrian tracking in real scenarios," Jul. 2025. [Online]. Available: http://arxiv.org/abs/2507.02479.

[18] L. Yang, L. Li, X. Xin, Y. Sun, Q. Song, and W. Wang, "Large-scale person detection and localization using overhead fisheye cameras," 2020. [Online]. Available: https://arxiv.org/abs/2307.08252.

[19] Y. Li, Z. Xiao, L. Yang, *et al.*, "Attmot: Improving multiple-object tracking by introducing auxiliary pedestrian attributes," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, pp. 5454–5468, 3 2025, ISSN: 21622388. DOI: 10.1109/TNNLS.2024.3384446. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10508509.

[20] A. Elaoua, M. Nadour, L. Cherroun, and A. Elasri, "Real-time people counting system using yolov8 object detection," *Proceedings - 2023 2nd International Conference on Electronics, Energy and Measurement, IC2EM 2023*, 2023. DOI: 10.1109/IC2EM59347.2023.10419684. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10419684.

[21] R. B. Grady, *Practical software metrics for project management and process improvement*. Prentice-Hall, Inc., 1992.

[22] I. Jacobson, *The unified software development process*. Pearson Education India, 1999.

[23] P. B. Kruchten, "The 4+ 1 view model of architecture," *IEEE software*, vol. 12, no. 6, pp. 42–50, 1995.

[24] project-jqh0m, *Pedestrian detection dataset*, https://universe.roboflow.com/project-jqh0m/pedestrian-otve0, visited on 2026-04-29, Apr. 2026. [Online]. Available: https://universe.roboflow.com/project-jqh0m/pedestrian-otve0.

[25] Rajarshi, *Pedestrian/human detection dataset*, https://www.kaggle.com/datasets/rajarshi2712/pedestrianhuman-detection, visited on 2026-04-29. [Online]. Available: https://www.kaggle.com/datasets/rajarshi2712/pedestrianhuman-detection.

[26] W. Hou, *MOT17 dataset*, https://www.kaggle.com/datasets/wenhoujinjust/mot-17, visited on 2026-04-29, 2024.

[27] Citypersons conversion, *CityPersons dataset*, https://universe.roboflow.com/citypersons-conversion/citypersons-woqjq, visited on 2026-04-29, 2022.